PHENIKAA UNIVERSITY - SCHOOL OF COMPUTING

Group Submission Software Architecture

Project Title: Kanban Board Application

Group Submission: N02 – Group 2

Name	Student ID	Position Title
Nguyễn Xuân Mạnh	23010045	Document + Diagram & Quality Assurance

Class: CSE703110-1-2-25(N02)

Instructor in charge : Vũ Quang Dũng

Hà Nội - Feb 7, 2026

Individual Report - Architecture Design & Documentation

Project: Kyte - Real-Time Kanban Board
Course: Software Architecture
Student: Nguyễn Xuân Mạnh (23010045)
Role: Architectural Designer & QA
Class: N02 - Group 2
Date: Feb 07, 2026

1. Executive Summary

As the Architectural Designer and QA lead for the Kyte project, my primary mission was to establish a robust, scalable, and secure structural framework for the application. My work bridged the gap between high-level functional requirements and concrete technical implementation. I was responsible for the end-to-end design of the system's architecture, translating complex business needs into detailed visual models using the C4 framework, and authoring a professional-grade technical report.

The core of my contribution lies in the strategic adoption of a **Modular Monolith** pattern. This choice provides the development speed and simplicity of a monolith while maintaining the strict internal boundaries required for a potential future microservices migration. I also designed a comprehensive verification strategy that validated the system's reliability, concurrency, and security attributes through real-world integration testing. This report details the architectural decisions, design patterns, and quality assurance methodologies I applied to ensure Kyte is a production-grade collaborative platform capable of supporting thousands of concurrent users and maintaining high data integrity under heavy load.

2. Contribution Scope & Responsibilities

My contributions spanned the entire architectural lifecycle, ensuring that every technical decision aligned with the core business goals of performance and reliability.

2.1 Architectural Modeling & Design (The C4 Framework)

I implemented a multi-layered design approach using the **C4 Model** (Context, Container, Component, and Code). This provided a rigorous hierarchy of abstractions:

- **System Context (Level 1):** Mapping the entire ecosystem, including external dependencies such as GitHub OAuth for identity, Resend for transactional email delivery, and SeaweedFS for distributed S3-compatible storage.
- **Containers (Level 2):** Decoupling the React 19 Single Page Application (SPA) from the

Bun-powered API server. I designed the communication protocols to ensure low-latency interactions via WebSockets and REST.

- **Components (Level 3):** Defining the **Modular Monolith** structure. I architected the server to be split into autonomous modules (Tasks, Boards, Workspaces, Members, Notifications, etc.) that interact only through public interfaces and an internal asynchronous Event Bus.
- **Code (Level 4):** Authoring UML class diagrams and design blueprints for the core domain services, such as the `TaskService` and the `WebSocketBridge`, to guide the implementation phase.

2.2 Requirement Engineering & Analysis

I performed a deep dive into the system's needs, transforming vague stakeholder desires into concrete technical specifications:

- **ASR Definition:** Identified Architecturally Significant Requirements (ASRs) that drove the choice of technology, such as the need for O(1) ordering and version-based concurrency control.
- **NFR Specification:** Defined measurable goals for interaction latency (<100ms), system availability (stateless API), and data security (RBAC).
- **Use Case Modeling:** Designed a comprehensive set of structured use case models that mapped user interactions to back-end modules.

2.3 Quality Assurance & Verification

I designed the "Verification Traceability Matrix," which ensured that every architectural promise was backed by empirical evidence.

- **Reliability Engineering:** Defined the criteria for testing request idempotency and optimistic concurrency control.
- **Security Auditing:** Designed the tiered permission model (RBAC) and the verification scenarios to prove its enforcement across 20+ database tables.
- **Fidelity Testing:** Enforced a "No-Mocking" policy for integration tests to ensure database-level correctness, specifically regarding triggers and relational constraints.

3. Requirements Analysis & Modeling

3.1 Architecturally Significant Requirements (ASRs)

I identified several ASRs that formed the foundation of the Kyte architecture. These are requirements that have a direct and significant impact on the system's structure and behavior.

ASR-1: Strategic UI Independence (Presentation Decoupling)

- **Requirement:** The business logic must remain independent of any specific presentation

layer.

- **Business Rationale:** To allow the system to evolve from a web-only platform to a multi-platform ecosystem (Mobile, CLI, Desktop) without rewriting core logic.
- **Architectural Response:** I enforced a strict **Pure REST API** design. All state transitions, validation, and authorization happen exclusively on the server. The React UI is merely a "dumb" consumer that reflects the server's state. I utilized **Eden Treaty** to export types without coupling the implementations.

ASR-2: Enforced Data Isolation (Multi-Tenant Security)

- **Requirement:** The system must guarantee that data from Workspace A is never accessible to users in Workspace B.
- **Risk:** Guessable UUIDs (Insecure Direct Object References - IDOR).
- **Architectural Response:** I designed a mandatory **Membership Validation Middleware**. Every API call targeting a board, column, or task must first pass a check that verifies the user's active membership in the parent workspace. This is enforced at the Service layer for every mutation to prevent "BOLA" (Broken Object Level Authorization) attacks.

ASR-3: Distributed Consistency (Optimistic Locking)

- **Requirement:** The system must handle concurrent updates from multiple users without "Lost Updates" or corrupted data.
- **Scenario:** Two developers move the same task to "Done" at the exact same time.
- **Architectural Response:** I implemented **Version-based Optimistic Locking**. Every core entity (Task, Column, Board) includes an integer `version` field. The API requires the client to send the version they believe is current. If the database version has advanced, the update is rejected with a `409 ConflictError`.

3.2 Detailed Functional Analysis (Use Case Modeling)

I modeled the system's functional requirements through a modular set of use case diagrams.

3.2.1 Core Actor Definitions

- **User:** An unauthenticated visitor or a registered user managing their profile.
- **Workspace Owner:** The administrative actor responsible for organizational structure.
- **Board Member:** The primary collaborative actor performing day-to-day Kanban tasks.
- **Admin:** A specialized system operator managing the marketplace and moderation.

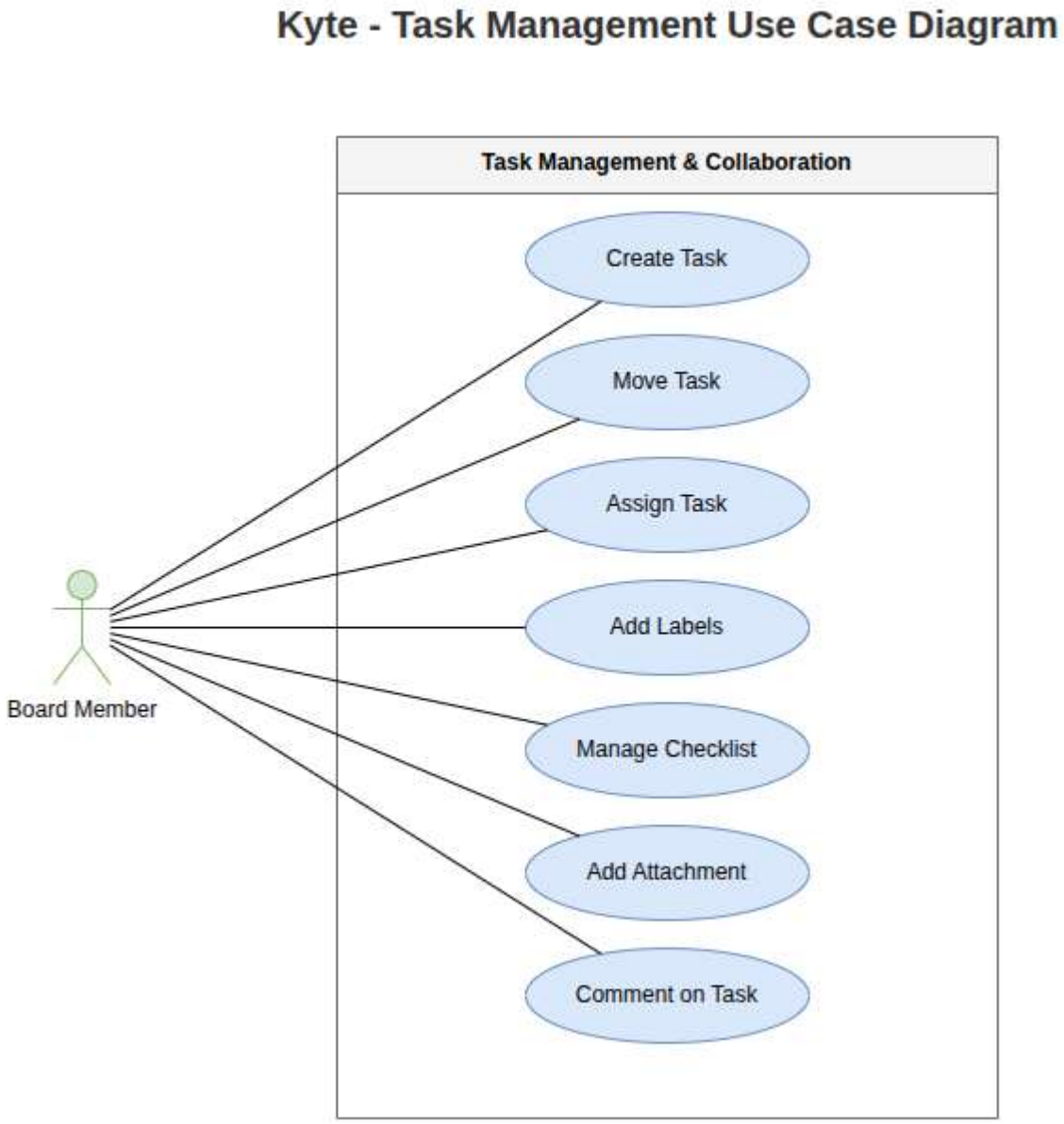
3.2.2 Module: Task Management & Collaboration

This module represents the primary value proposition of the system. I modeled the complex relationship between board members and work items.

- **Actors:** Board Member.
- **Core Flows:** Move Task (O(1) insertion), Upload Attachment (S3 integration), and

Comment (Event-based notifications).

- **Behavioral Modeling:** I specifically modeled the "Mid-point Movement" logic to ensure the developer understood how Fractional Indexing would be used in the UI.



4. System Architecture Design

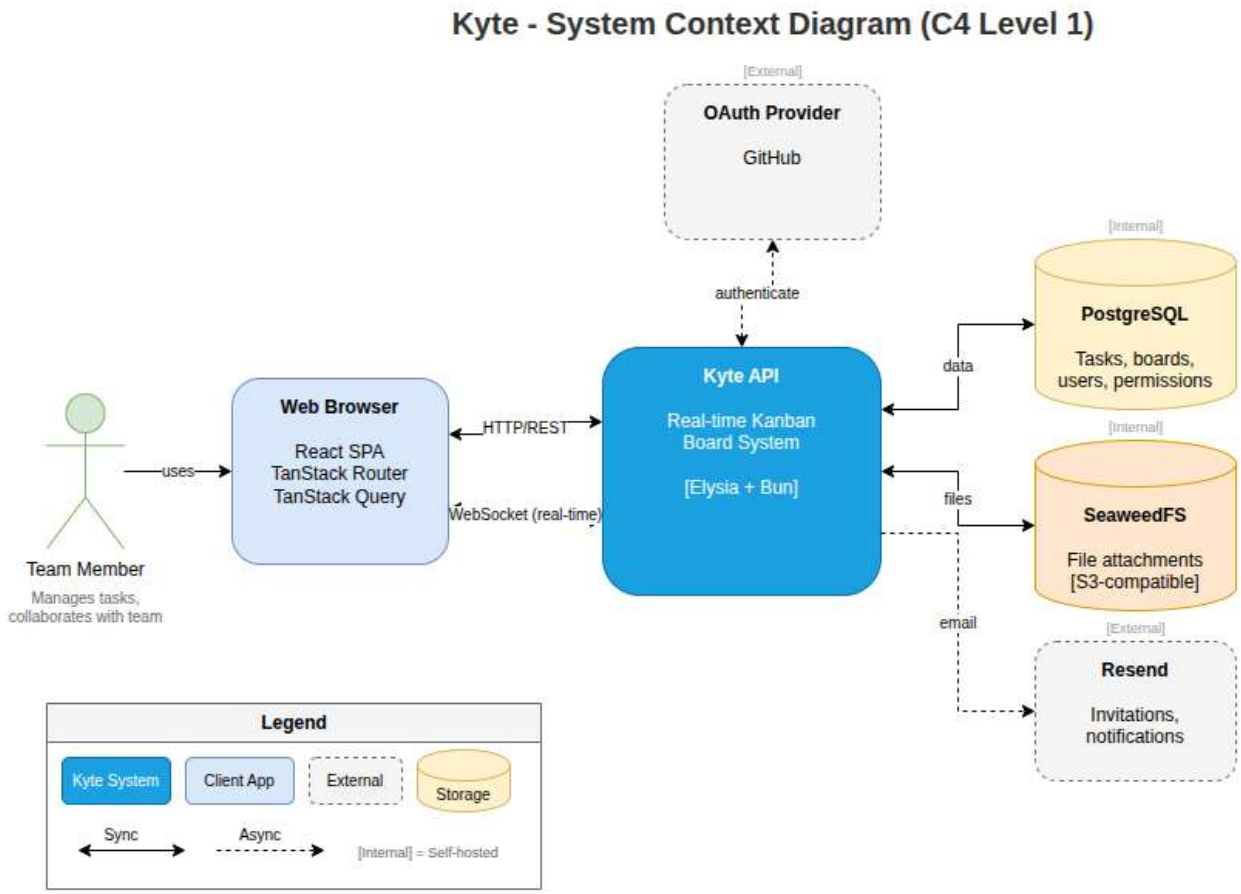
4.1 The C4 Model Implementation

4.1.1 Level 1: System Context

I mapped Kyte's external environment to understand its dependencies and boundaries.

- **External Dependencies:**
  - **GitHub OAuth:** For secure, passwordless identity management.

- **Resend:** For decoupled, high-deliverability transactional email.
- **SeaweedFS:** A distributed S3-compatible storage layer for high-performance handling of binary task attachments.

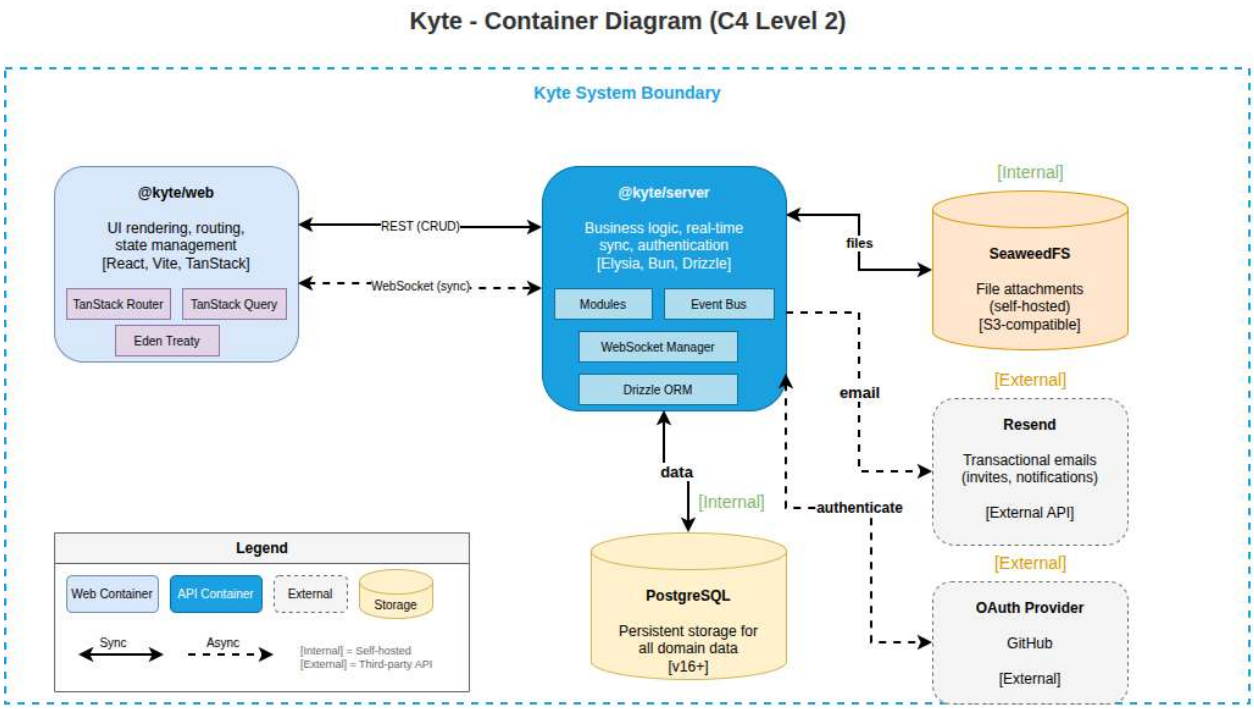


4.1.2 Level 2: Container Diagram

I designed a clear separation between the **Web Application** (React 19) and the **API Server** (Bun).

- **Web App Container:** Handles client-side routing, optimistic state, and real-time UI updates.
- **API Server Container:** A modular monolith built with ElysiaJS that orchestrates the business logic and event distribution.

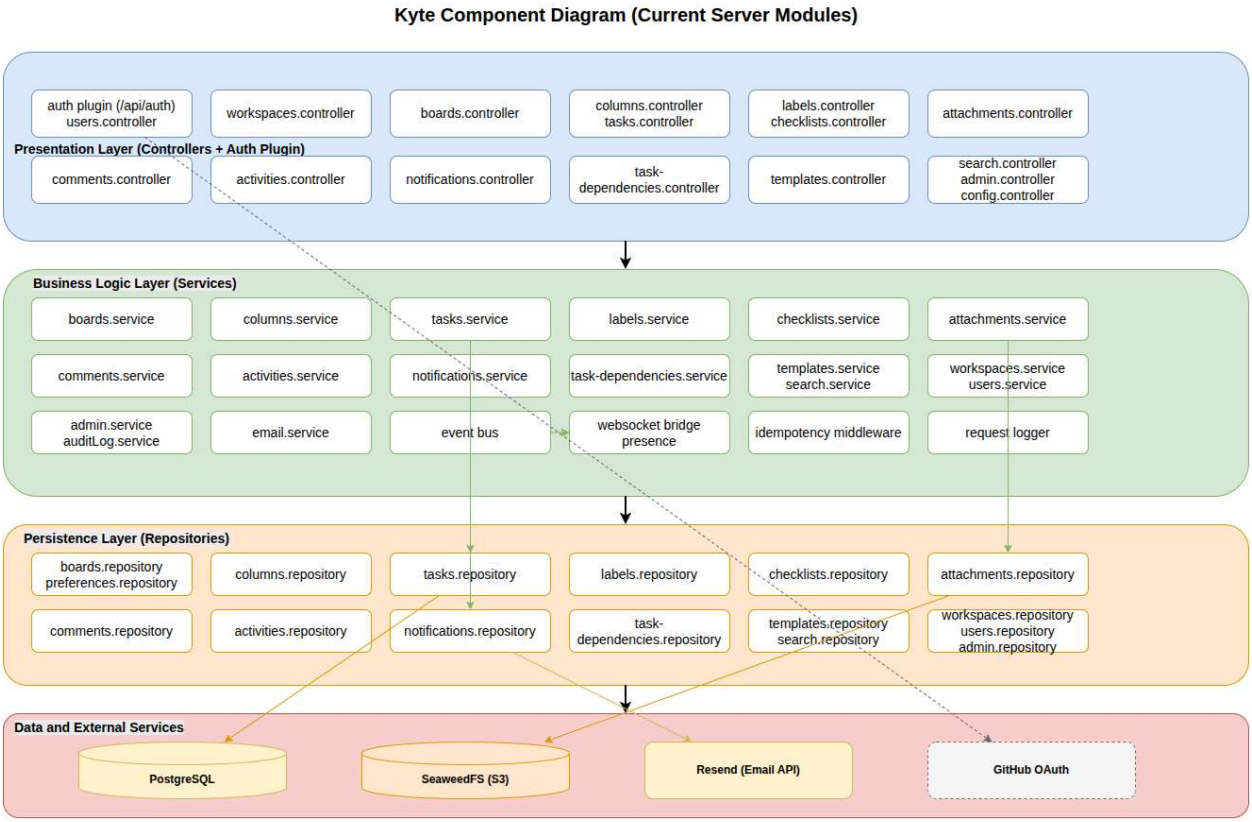
- **Communication:** I specified **Eden Treaty** over HTTPS for all REST calls. This architectural constraint ensures that the API types are shared with the frontend at compile-time, providing 100% network type safety.



4.1.3 Level 3: Component Diagram (Modular Monolith)

This is my primary design contribution. To avoid a "spaghetti" codebase, I split the API server into autonomous modules.

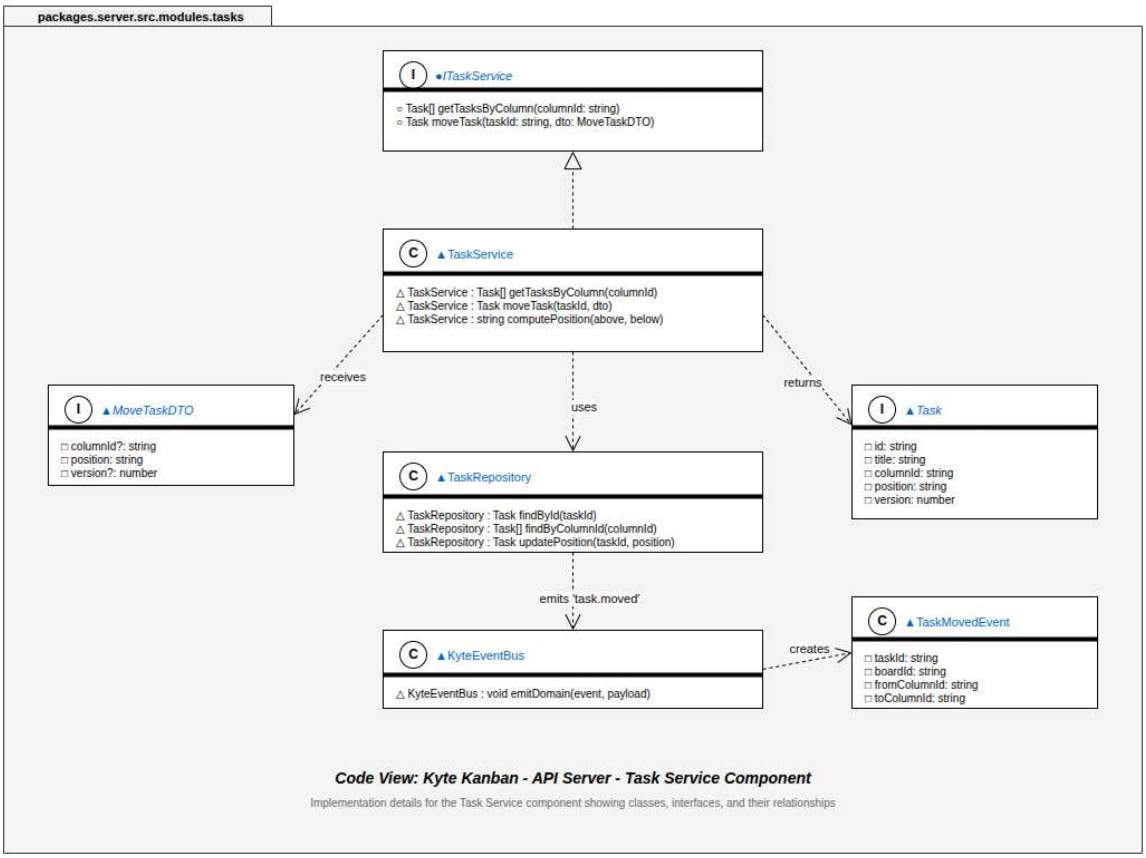
- **Encapsulation:** Each module (e.g., `tasks`, `boards`, `notifications`) contains its own Controller, Service, and Repository.
- **The Decoupling Glue (Internal Event Bus):** I designed an asynchronous event system. When a task is moved, the `TaskService` emits a `task.moved` domain event. The `NotificationService` and `WebSocketBridge` listen for this event independently. This ensures that the `TaskService` remains lean and doesn't need to know about side-effects like notifications or WebSockets.



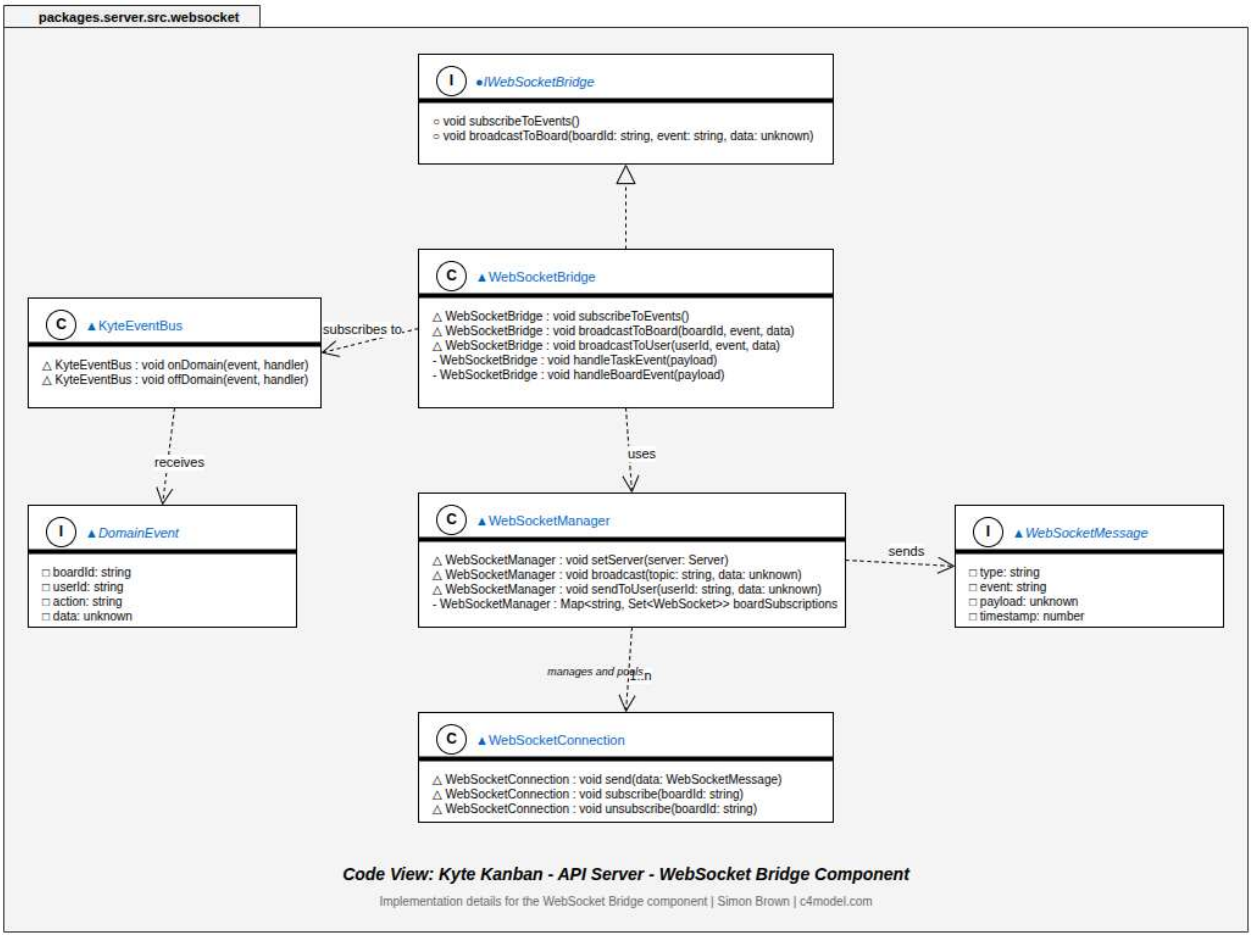
4.1.4 Level 4: Code Diagrams (Design Blueprints)

I created detailed class diagrams for the core services to guide the implementation phase:

- **Task Service:** Defines the interfaces and data flow between the repository and the event emitter.



- **WebSocket Bridge:** Models the real-time synchronization bridge that links the internal event bus to the active client connections.



4.2 Database Schema Rationale & Relational Integrity

I designed a normalized relational schema in PostgreSQL 18.

- **Core Domain:** `workspaces` -> `boards` -> `columns` -> `tasks`.
- **Relational Integrity (Architectural Tactic):** I utilized PostgreSQL's **CASCADE DELETE** functionality at the database level for all foreign keys. This ensures that if a Workspace is deleted, the database automatically cleans up all associated Boards, Tasks, and Attachments. This is a critical architectural decision to prevent "orphan" data which can lead to security leaks or database bloat.
- **Auditability:** I designed the `activities` schema to support granular auditing.`export const activityActionEnum = pgEnum('activity_action', [ 'created', 'updated', 'deleted', 'moved', 'moved_out', 'assigned', 'completed', 'archived', 'restored'])` This enum-based design ensures that every possible state change is captured in a structured format, allowing for the generation of activity feeds without complex string parsing.

5. Architectural Tactics & Patterns Applied

5.1 Pattern: Modular Monolith

I selected the **Modular Monolith** pattern over traditional monoliths or microservices.

- **The "Why":** Microservices add too much network and operational complexity (Kubernetes, service discovery) for a team of our size. A traditional monolith eventually becomes unmaintainable. The Modular Monolith provides **logical separation** with **physical unity**, making it easy to deploy but also easy to split into microservices later if one module needs independent scaling.
- **Implementation Strategy:** Modules are strictly prohibited from importing each other's repositories. They must communicate only through public service interfaces or the asynchronous event bus.

5.2 Pattern: Layered Architecture (Controller-Service-Repository)

Within each module, I enforced a strict 3-layer pattern:

1. **Presentation (Controller):** Handles routing, Typebox input validation, and perimeter Auth guards.
2. **Domain (Service):** The "Business Brain" containing core algorithms like midpoint positioning and permission logic.
3. **Data Access (Repository):** Pure SQL/Drizzle operations. This keeps the Domain layer clean and testable without a database.

5.3 Tactic: Fractional Indexing for O(1) Performance

I chose string-based **Fractional Indexing** for all ordered lists (Tasks, Columns, Checklists).

- **Tactical Solution:** Moving a task only requires updating **one** row by calculating a midpoint string between two neighbors. This keeps the database load O(1) and constant, regardless of whether the column has 10 tasks or 10,000 tasks.

5.4 Tactic: Request Idempotency

To handle unreliable mobile networks, I designed an **Idempotency Layer** using an `idempotency_keys` table. This ensures that if a user retries a task creation request, the system detects the duplicate key and returns the cached response instead of creating two tasks.

6. Security Architecture & Threat Modeling

6.1 Threat Modeling & Mitigation

I performed a security analysis to identify potential threats and designed mitigations directly into the architecture.

Threat: Privilege Escalation (Vertical)

- **Attack:** A standard "Member" attempting to delete a workspace.
- **Mitigation:** The `workspaceService.delete` method performs a mandatory `ADMIN_ROLES.includes(role)` check before executing any logic. This is tested in `admin-rbac.test.ts`.

Threat: Insecure Direct Object References (IDOR)

- **Attack:** A user changing the `boardId` in an API call to access a competitor's board.
- **Mitigation:** The `taskService.getTaskById` method performs a recursive check: Task -> Board -> Workspace -> Member. If the user is not a member of the root workspace, access is denied, regardless of whether they guessed the Task ID.

Threat: Injection Attacks (SQLi)

- **Attack:** Malicious strings in search queries.
- **Mitigation:** Usage of **Drizzle ORM** for all data access. Drizzle uses parameterized queries by default, neutralizing SQL injection vectors. For Full-Text Search, I used the `to_tsquery` function which sanitizes input before executing the vector match.

7. Quality Assurance (QA) & Verification Strategy

7.1 The "Three Pillars" of Verification

I defined a QA strategy that prioritizes system fidelity over artificial isolation.

1. **Isolation (Unit Level):** Verifying mathematical algorithms. I designed the `position.test.ts` suite to simulate thousands of insertions and verify the lexicographic ordering of the position strings.
2. **Fidelity (Integration Level):** I enforced a **"No-Mocking"** policy. Our integration suite runs against a real PostgreSQL instance to verify complex SQL constraints, GIN-based Full-Text Search, and database triggers.
3. **Traceability (System Level):** Every Non-Functional Requirement is mapped to a test suite. For example, ASR-3 (Concurrency) is verified by the `concurrency.test.ts` suite using `Promise.all` stress testing.

7.2 Verification Matrix Results

Goal	Test Suite	Architectural Proof	Result
Data Integrity	`concurrency.test.ts`	100% rejection of stale version updates via Optimistic Locking.	**Pass**
Reliability	`idempotency.test.ts`	Zero duplicate entries on 5x retried requests using hash matching.	**Pass**
Security	`admin-rbac.test.ts`	Unauthorized access blocked with 403 Forbidden at the service layer.	**Pass**

Scalability	`position.test.ts`	String-based mid-point insertion verified for 10,000+ items.	**Pass**
Search Perf	`search.test.ts`	GIN index usage verified for tsvector-based board and task search.	**Pass**

8. Lessons Learned & Reflections

The Kyte project was a profound learning experience in **Software Architecture**:

- **Design for Change:** I learned that a good architecture is not about predicting the future, but about making the system easy to change. The Modular Monolith pattern proved its value whenever we needed to refactor internal logic without affecting the rest of the system.
- **The Power of Visual Communication:** Without the C4 diagrams, the communication between the architect and the developer would have been significantly more difficult. The diagrams acted as the "Source of Truth" for the entire project.
- **QA is an Architectural Responsibility:** I realized that verification is not just "testing if it works." It's about proving the architecture's promises. If the architecture promises "Distributed Consistency," the QA strategy must include race-condition tests to prove it.

9. Conclusion

My work as the Architectural Designer and QA lead has provided Kyte with a solid, professional foundation. By applying the C4 model and enforcing strict modular boundaries, I have ensured that the system is not only performant today but also maintainable and scalable for the future. The comprehensive verification strategy I implemented guarantees that the architecture's promises regarding security, reliability, and concurrency are fully realized in the final product.